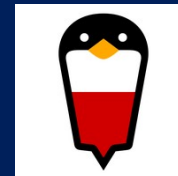




Introduction to CUDA

Massimiliano Ghilardi
Lucca, 2026-06-27



1.0 Introduction to CUDA

Learn how to use GPUs parallelism for improving your own software performance.

Fundamentals, architecture, programming paradigms and optimizations for NVIDIA GPU development.

Hardware and software behind the A.I. craze – how does it work?



1.1 CUDA programming model

At first look, appears extremely similar to classic C/C++

“only” some new keywords: `__global__` `__device__` `__shared__` `__constant__` ...

Appearance can be misleading

SIMT: the same code is executed in parallel by thousands of threads

synchronization is manual, and very costly

memory hierarchy managed **explicitly** by programmer

- local variables → registers
- `__shared__` variables → shared memory i.e. cache
- constants → constant memory, cached
- everything else → global memory



1.2 – CUDA execution model

Parallelism is managed explicitly

threads are grouped in “blocks”: usually 32...1024 per block – HW prefers $32 \cdot N$ (warp)

blocks are grouped in a “grid” – all blocks in a grid must have the same size.

block size $\{x,y,z\}$ and grid size $\{x,y,z\}$ are specified by programmer

GPU has N “Streaming Multiprocessors” = SM. example: 170 SM on RTX 5090

each SM executes one or more blocks i.e. up to ~ 1024 threads

CUDA entry point:

any function annotated `__global__`

it can call only functions annotated `__device__`

each thread gets a thread ID (inside its block) and a block ID (inside the grid)

used to select the data to work on



1.3 – minimal CUDA example

```
__global__ void KernelSum(const float* aIn1, const float* aIn2, float* aOut, unsigned aSize)
{
    unsigned myPos = blockIdx.x * blockDim.x + threadIdx.x;
    if (myPos < aSize) {
        aOut[myPos] = aIn1[myPos] + aIn2[myPos];
    }
}
```

```
#define BLOCK_SIZE 512
/*__host__*/ void CudaSum(const float* aIn1, const float* aIn2, float* aOut, unsigned aSize)
{
    dim3 myThreads(BLOCK_SIZE);
    dim3 myGrid((aSize + BLOCK_SIZE - 1) / BLOCK_SIZE);
    KernelSum<<<myGrid, myThreads>>>(aIn1, aIn2, aOut, aSize);
}
```



1.3 – minimal CUDA example

```
__global__ void KernelSum(const float* aIn1, const float* aIn2, float* aOut, unsigned aSize)
{
    unsigned myPos = blockIdx.x * blockDim.x + threadIdx.x; // 1d thread position in block & grid
    if (myPos < aSize) {
        aOut[myPos] = aIn1[myPos] + aIn2[myPos];
    }
}
```

```
#define BLOCK_SIZE 512
/*__host__*/ void CudaSum(const float* aIn1, const float* aIn2, float* aOut, unsigned aSize)
{
    dim3 myThreads(BLOCK_SIZE);
    dim3 myGrid((aSize + BLOCK_SIZE - 1) / BLOCK_SIZE);
    KernelSum<<<myGrid, myThreads>>>(aIn1, aIn2, aOut, aSize);
}
```



1.3 – minimal CUDA example

```
__global__ void KernelSum(const float* aIn1, const float* aIn2, float* aOut, unsigned aSize)
{
    unsigned myPos = blockIdx.x * blockDim.x + threadIdx.x; // 1d thread position in block & grid
    if (myPos < aSize) { // always check bounds: block & grid may not match problem size exactly
        aOut[myPos] = aIn1[myPos] + aIn2[myPos];
    }
}
```

```
#define BLOCK_SIZE 512
/*__host__*/ void CudaSum(const float* aIn1, const float* aIn2, float* aOut, unsigned aSize)
{
    dim3 myThreads(BLOCK_SIZE);
    dim3 myGrid((aSize + BLOCK_SIZE - 1) / BLOCK_SIZE);
    KernelSum<<<myGrid, myThreads>>>(aIn1, aIn2, aOut, aSize);
}
```



1.3 – minimal CUDA example

```
__global__ void KernelSum(const float* aIn1, const float* aIn2, float* aOut, unsigned aSize)
{
    unsigned myPos = blockIdx.x * blockDim.x + threadIdx.x; // 1d thread position in block & grid
    if (myPos < aSize) { // always check bounds: block & grid may not match problem size exactly
        aOut[myPos] = aIn1[myPos] + aIn2[myPos]; // useful work
    }
}
```

```
#define BLOCK_SIZE 512
/*__host__*/ void CudaSum(const float* aIn1, const float* aIn2, float* aOut, unsigned aSize)
{
    dim3 myThreads(BLOCK_SIZE);
    dim3 myGrid((aSize + BLOCK_SIZE - 1) / BLOCK_SIZE);
    KernelSum<<<myGrid, myThreads>>>(aIn1, aIn2, aOut, aSize);
}
```



1.3 – minimal CUDA example

```
__global__ void KernelSum(const float* aIn1, const float* aIn2, float* aOut, unsigned aSize)
{
    unsigned myPos = blockIdx.x * blockDim.x + threadIdx.x; // 1d thread position in block & grid
    if (myPos < aSize) { // always check bounds: block & grid may not match problem size exactly
        aOut[myPos] = aIn1[myPos] + aIn2[myPos]; // useful work
    }
}
```

```
#define BLOCK_SIZE 512
/*__host__*/ void CudaSum(const float* aIn1, const float* aIn2, float* aOut, unsigned aSize)
{
    dim3 myThreads(BLOCK_SIZE); // choose block size
    dim3 myGrid((aSize + BLOCK_SIZE - 1) / BLOCK_SIZE); // choose grid size
    KernelSum<<<myGrid, myThreads>>>(aIn1, aIn2, aOut, aSize); // launch kernel
}
```



LIVE DEMO



2.1 – CUDA mechanical sympathy

Hardware is very different from CPUs

FASTEST	local variables	→ registers (256k * 4bytes per SM) cannot take their address
FAST	__shared__ variables	→ shared memory (48kB per SM) it's actually L1 cache
FAST	constants	→ constant memory (64kB) cached
AVERAGE	__syncthreads()	→ synchronize all threads in the same block
SLOW	local variables	→ local memory (= cached global memory)
SLOW	global memory, sequential read/write	
VERY SLOW	global memory, random read/write	
SLOWEST	synchronize all threads in a whole grid, or cudaStreamSynchronize()	
SLOWEST	host → cuda memory copy	
SLOWEST*	cuda → host memory copy	



2.2 – CUDA main optimizations

1. store data in local variables – **critical**
2. use `__shared__` memory to exchange data between threads
3. minimize the use of global memory
 - a single read/write takes as long as 100-200 arithmetic operations on registers
 - access consecutive addresses from consecutive threads
 - prefer sequential read + random write to viceversa
4. avoid thread divergence i.e. threads taking different 'if (...)' branches
5. minimize synchronizations – no need to synchronize between kernel executions
6. minimize host $\rightarrow$ cuda and cuda $\rightarrow$ host memory copies
7. kernel execution overhead is $\sim 3\mu\text{s}$: launch $\sim 1\mu\text{s}$, run $\geq 1\mu\text{s}$, synchronize $\sim 1\mu\text{s}$



2.3 – CUDA ideal kernel

The “ideal” kernel:

- performs long ($\geq 15\mu\text{s}$) independent computations ($\geq 10\text{k}$ threads) on many local variables
- synchronizes as little as possible – or not at all
- carefully uses `__shared__` memory
- very rarely uses global memory
- reads/writes small types in global memory:
INT8, INT16, FP8, FP16, BFLOAT16 rather than FLOAT32
- avoids FLOAT64 – extremely slow on consumer GPUs



2.4 – CUDA libraries

- linear algebra: cuBLAS level 1 (vector-vector), 2 (vector-matrix), 3 (matrix-matrix), NVBLAS can use Tensor Core HW – A.I. involves a huge number of matrix multiplications
- fourier transform: cuFFT
- random generator: cuRAND
- linear systems: cuSOLVER
- sparse matrix: cuSPARSE
- utilities: CUB, Thrust, GPUDirect (DMA from/to disk), CUDA on WSL, PTX assembly tools
- 2D image processing: NPP, nvJPEG
- debugging and profiling: CUDA-GDB, nvvp, nsight
- python wrappers/alternatives: CUDA Python, PyTorch, TensorFlow



2.5 – CUDA installation

- Debian stable and testing

```
apt install built-essential nvidia-driver-full nvidia-smi \  
nvidia-cuda-toolkit nvidia-cuda-toolkit-gcc
```

use a kernel < 7.0.0: compiling nvidia kernel module 550.x currently fails on kernels >= 7.0.0

- Ubuntu 18.04 ... 26.04

```
apt install ubuntu-drivers-common
```

```
ubuntu-drivers devices # choose a driver version among the listed ones
```

```
apt install built-essential nvidia-driver-NNN nvidia-utils-NNN libnvidia-gl-NNN \  
nvidia-cuda-toolkit libglvnd-dev
```

- reboot if installed a new kernel, then run **nvidia-smi**
- compiled CUDA programs may depend on kernel & nvidia module versions



2.6 – CUDA documentation & resources

Find CUDA arch number for each GPU model

<https://arnon.dk/matching-sm-architectures-arch-and-gencode-for-various-nvidia-cards/>

CPU & GPU specs database

<https://www.techpowerup.com/cpu-specs/>

<https://www.techpowerup.com/gpu-specs/>

CUDA docs

<https://docs.nvidia.com/cuda/>

<https://developer.nvidia.com/cuda-toolkit-archive> # older toolkits

CUDA install guide – uses NVIDIA-provided packages, not your distribution 🥵

<https://docs.nvidia.com/cuda/cuda-installation-guide-linux/index.html>

NVCC compiler options

<https://docs.nvidia.com/cuda/cuda-compiler-driver-nvcc/index.html>



2.6 – Nominal HW speed

	max FLOAT32 operations/s	max RAM throughput
Intel Core i9 14900K + DDR5-5600	~2 TFLOPS (32 threads)	89.6 GB/s
AMD Ryzen Threadripper PRO 7965W + DDR5-5600	~3.5 TFLOPS (192 threads)	332.8 GB/s
NVIDIA GeForce RTX 3060 Ti GDDR6X	16.2 TFLOPS	608.3 GB/s
NVIDIA GeForce RTX 5090	104.8 TFLOPS	1790 GB/s
NVIDIA B300	154 TFLOPS (dual GPU)	8200 GB/s (dual GPU)



3.1 – Alternatives

ROCm HIP – by AMD, closely reproduces CUDA paradigm and libraries

<https://rocm.docs.amd.com/projects/HIP/en/latest/index.html>

<https://github.com/ROCm/HIP>

OpenCL – multivendor, higher level

<https://www.khronos.org/opencv/>

<https://github.com/khronos.org/>

oneAPI – by Intel, “open standard”

<https://oneapi.io/>

<https://github.com/oneapi-src>

ZLUDA – run CUDA code unmodified on AMD GPUs



Thank you

Questions?

